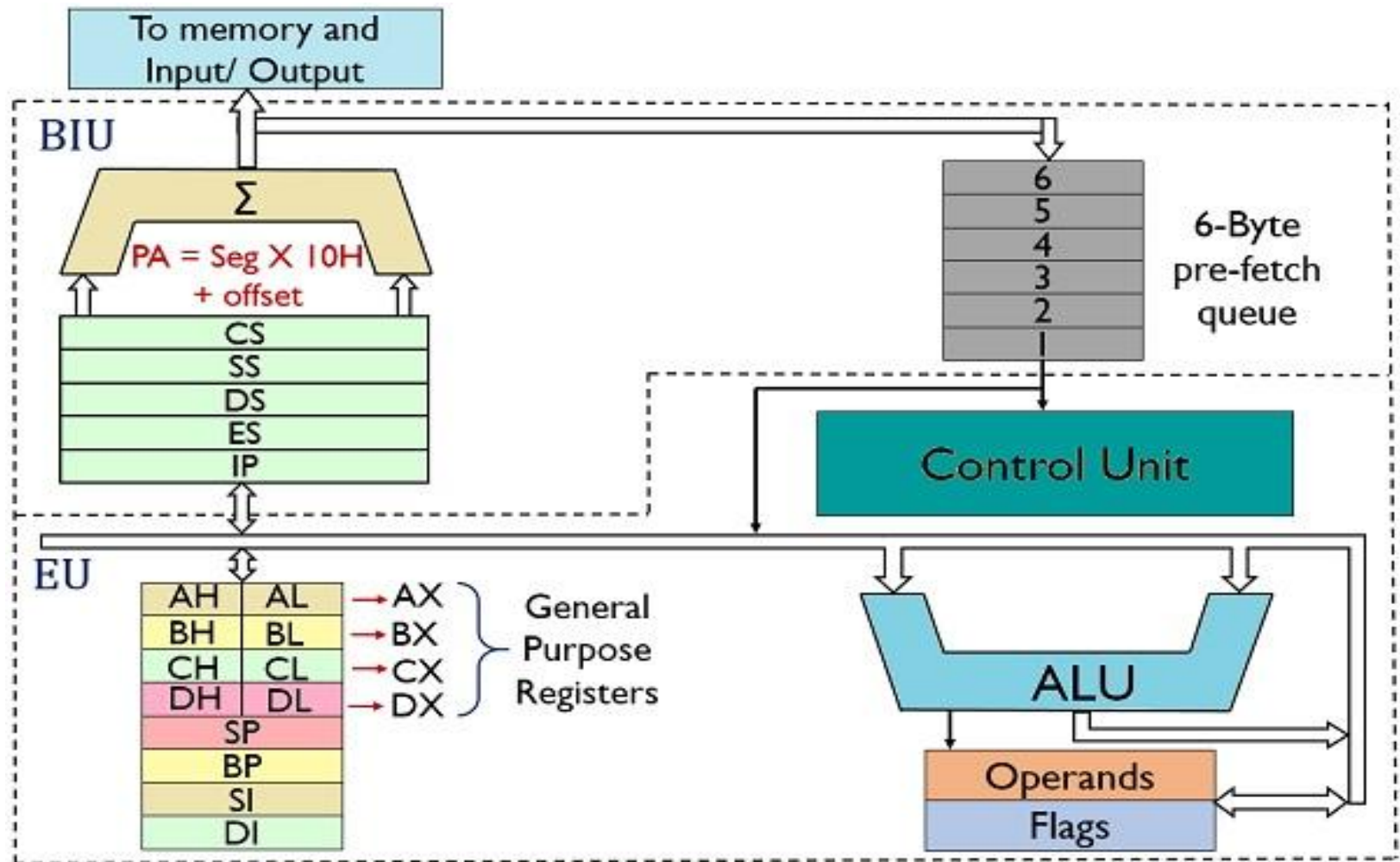


MICROPROCESSOR FUNDAMENTAL AND PROGRAMMING

8086

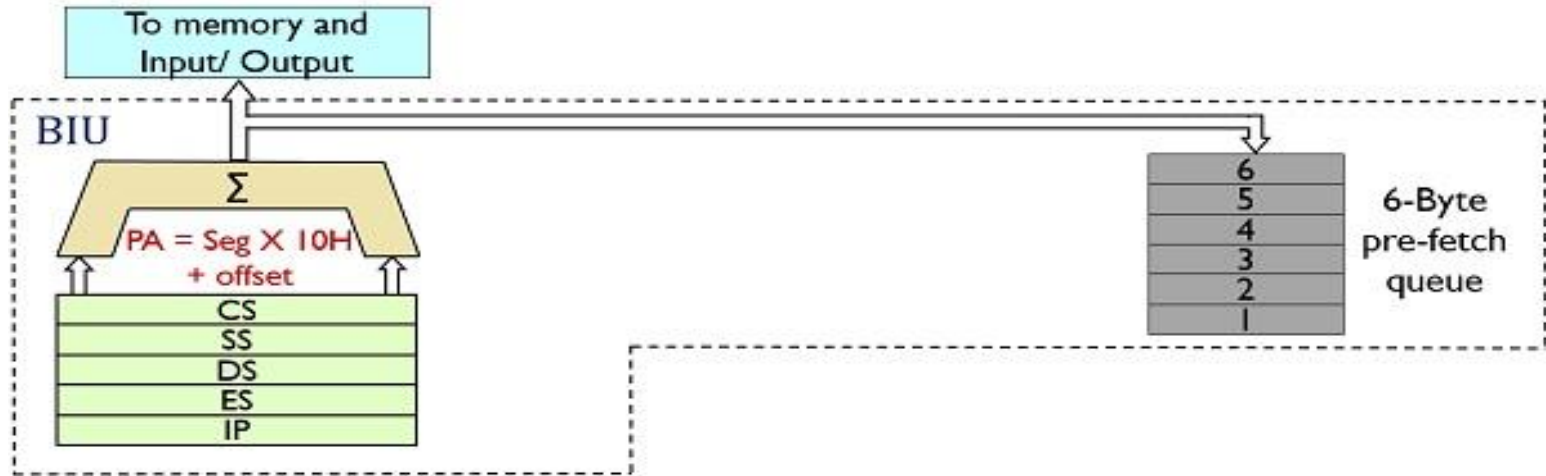
8086

- ENHANCED VERSION OF 8085
- 16-bit Microprocessor
- A 40 pin dual in line package
- It requires +5V power supply
- 20 address lines can access upto 2^{20} memory locations (1 MB) and 16 data lines
- 16 bit ALU
- 6 byte prefetch queue.



Block Diagram of 8086 Microprocessor

BUS INTERFACE UNIT(Contd)



- It generates the 20 bit physical address for memory access.
- It fetches instructions from the memory.
- It transfers data to and from the memory and I/O.
- Queuing of the prefetched instruction bytes.

- Prefetch Queue

- Concept of pipelining to provide parallelism between the BIU and EU
- While EU is busy in executing an instruction, the BIU can fetch the next instruction.
- First-in-first out 6 byte queue
- Also known as instruction queue.
- In case the next instruction is branch instruction the BIU flushes the queue and fills in with new address provided by branch instruction.

Segment Register and Instruction Pointer

- CS: Code Segment Register
- DS: Data Segment Register
- SS: Stack Segment Register
- ES: Extra Segment Register
- **Instruction Pointer:** contains the address of the next instruction that is to be executed.

Segmentation:

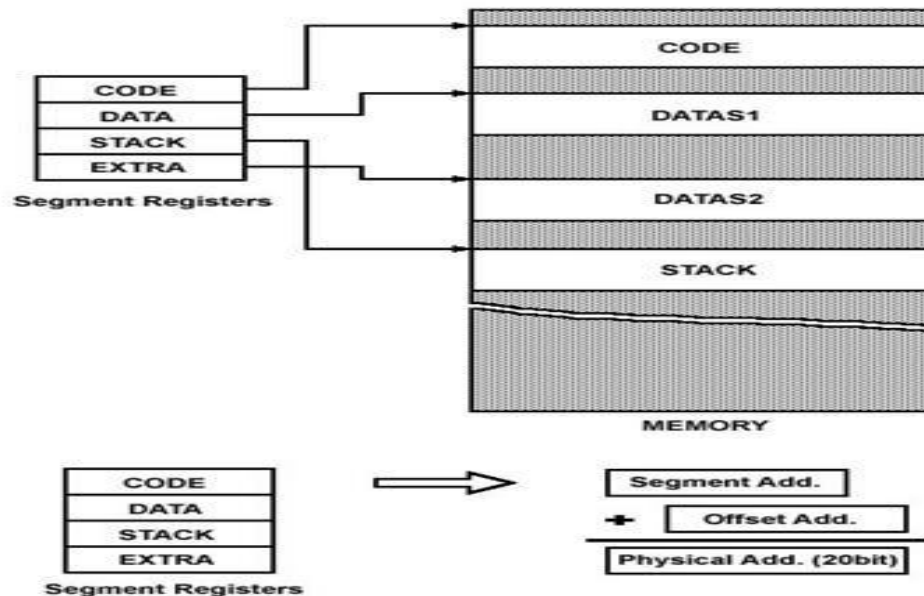
- The code segment is used to store the program instructions
- The data segment is used to store the data
- The stack segment provides the stack space for temporary values during the execution.
- The extra segment acts as another data segment.

The advantage of segmentation is that the three different components of a program code, data and stack reside in separate memory and thus changes in one can be made independent of others

Address Generation Circuit:

- The BIU has a Physical Address Generation Circuit.
- It generates the 20 bit physical address using Segment and Offset addresses using the formula:

Physical Address = Segment Address x 10H + Offset Address

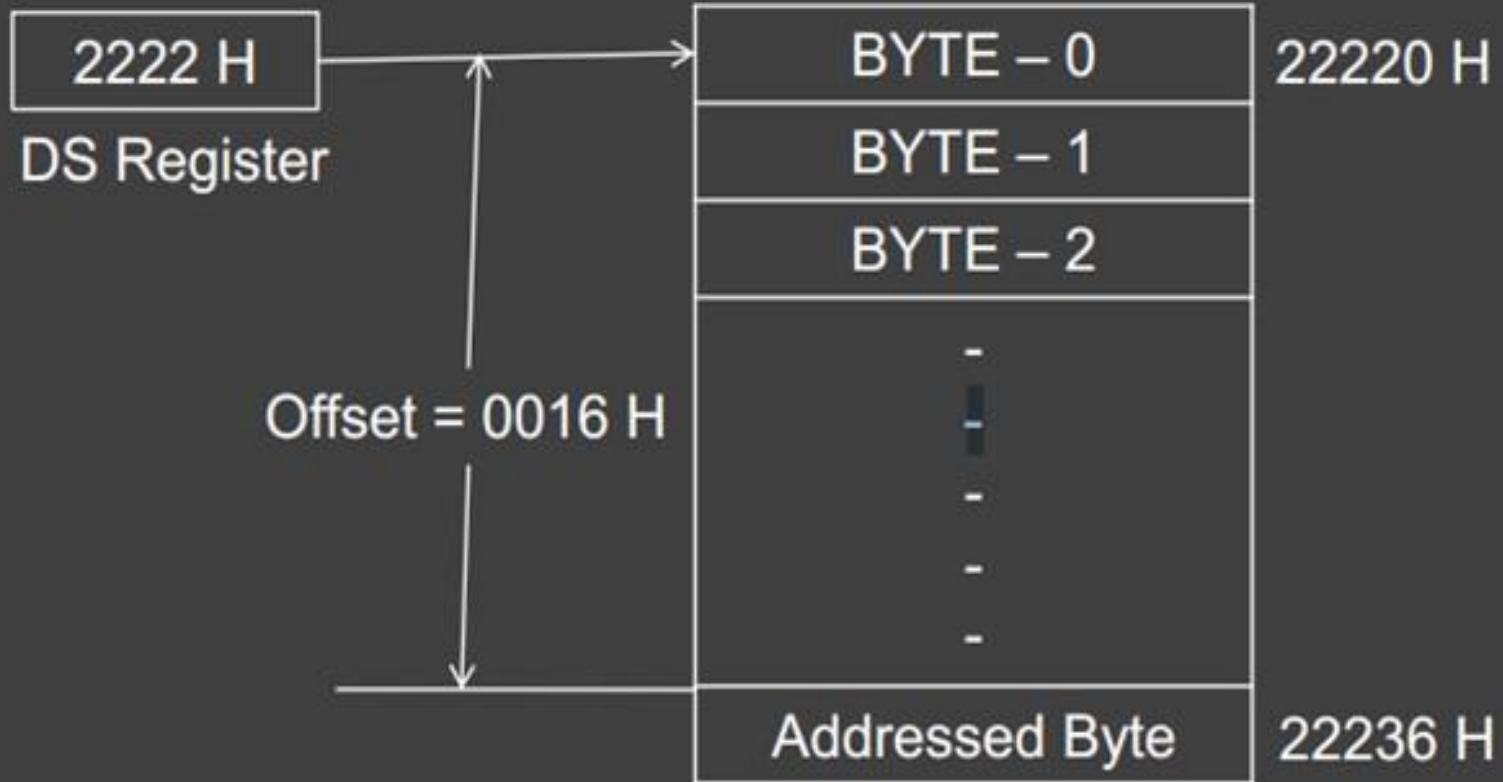


How is a 20-bit address obtained if there are only 16-bit registers?

- The 20-bit address of a byte is called its Physical Address.
- But, it is specified as a Logical Address. Logical address is in the form of:
Base Address : Offset
- Offset is the displacement of the memory location from the starting location of the segment

Example

- The value of Data Segment Register (DS) is 2222 H.
- To convert this 16-bit address into 20-bit, the BIU appends 0H to the LSBs of the address.
- After appending, the starting address of the Data Segment becomes 22220H.
- If the data at any location has a logical address specified as: 2222 H : 0016 H
- $PA = 22220\text{ H} + 0016\text{ H} = 22236\text{ H}$



Maximum size of segment

- All offsets are limited to 16-bits.
- It means that the maximum size possible for segment is $2^{16} = 65,535$ bytes (64 KB).
- The offset of the first location within the segment is 0000 H.
- The offset of the last location in the segment is FFFF H

Where to look offset

Segment	Offset Registers	Function
CS	IP	Address of the next instruction
DS	BX, DI, SI	Address of data
SS	SP, BP	Address in the stack
ES	BX, DI, SI	Address of destination data (for string operations)

Example

The value of Code Segment (CS) Register is 4042H and the value of different offsets is as follows:

BX: 2025H , IP: 0580H , DI: 4247H

Calculate the effective address of the memory location pointed by the CS register.

Example(Cont..)

Since the offset of the CS Register is the IP register.

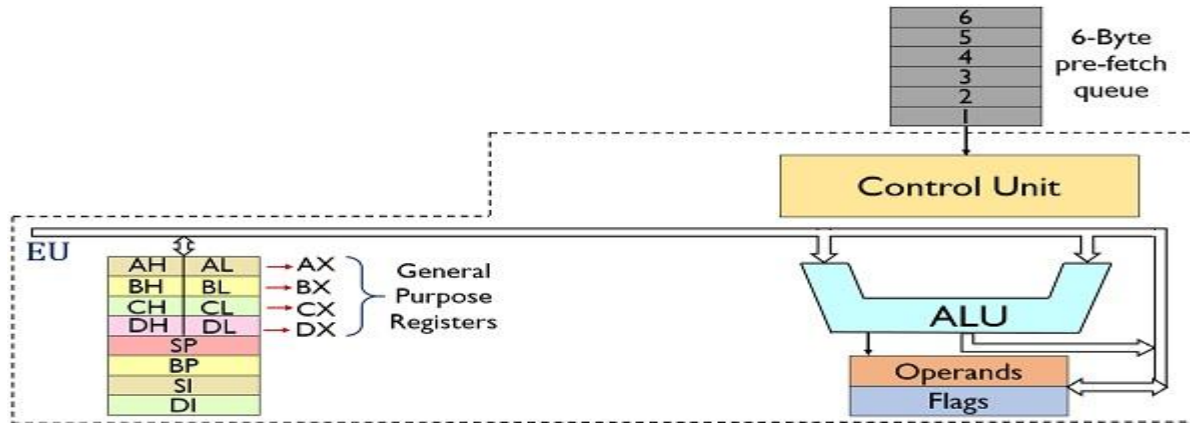
Therefore, the effective address of the memory location pointed by the CS register is calculated as follows:

$$\begin{aligned}\text{Effective address} &= \text{Base address of CS register} \times 10_H + \text{Address of IP} \\ &= 4042_H \times 10_H + 0580_H = (40420 + 0580)_H = 409A0_H\end{aligned}$$

What is the maximum and minimum size of the segment??

- Paragraph boundry- minimum size of the segment
- 16 bytes: Minimum
- 64KB:Maximum

EXECUTION UNIT



- It fetches instructions from the Queue in BIU, decodes and executes them.
- It performs arithmetic, logic and internal data transfer operations
- It sends request signals to the BIU to access the external module
- It handles all arithmetic and logical operations, like +, -, ×, /, OR, AND, NOT operations.

EXECUTION UNIT(Contd)

15	H	8	7	L	0
AX (Accumulator)					
AH			AL		
BX (Base Register)					
BH			BL		
CX (Used as a counter)					
CH			CL		
DX (Used to point to data in I/O operations)					
DH			DL		

AX - the Accumulator

BX - the Base Register
-XLAT

CX - the Count Register

DX - the Data Register
-Multiply, divide etc

- Normally used for storing temporary results
- Each of the registers is 16 bits wide (**AX, BX, CX, DX**)
- Can be accessed as either 16 or 8 bits AX, AH, AL

- **AX**

- Accumulator Register
- Preferred register to use in arithmetic, logic and data transfer instructions because it generates the shortest Machine Language Code
- Must be used in multiplication and division operations
- Must also be used in I/O operations

- **BX**

- Base Register
- Also serves as an address register

- **CX**

- Count register
- Used as a loop counter
- Used in shift and rotate operations

- **DX**

- Data register
- Used in multiplication and division
- Also used in I/O operations

Pointers and Index Registers

- Stack Pointer (SP) is a 16-bit register pointing to program stack
- Base Pointer (BP) is a 16-bit register pointing to data in stack segment. BP register is usually used for based, based indexed or register indirect addressing.
- Source Index (SI) is a 16-bit register. SI is used for indexed, based indexed and register indirect addressing, as well as a source data addresses in string manipulation instructions.
- Destination Index (DI) is a 16-bit register. DI is used for indexed, based indexed and register indirect addressing, as well as a destination data addresses in string manipulation instructions.

FLAG REGISTER

Bits	D ₁₅	D ₁₄	D ₁₃	D ₁₂	D ₁₁	D ₁₀	D ₉	D ₈	D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D ₀
Flag					O	D	I	T	S	Z		AC		P		CY

Flag Bit	Function (Status Flags)
S	After any operation if the MSB is 1, then it indicates that the number is negative. And this flag is set to 1
Z	If the result is zero, then only the Z flag is set
AC	When some arithmetic operations generates carry after the lower half and sends it to upper half, the AC will be 1
P	When result has even number of 1s, it will be set to 1, otherwise 0 for odd number of 1s
CY	This is carry bit. If some operations are generating carry after the operation this flag is set to 1
O	The overflow flag is set to 1 when the result of a signed operation is too large to fit.

Flag Bit	Function (Control Flags)
D	This is directional flag. This is used in string related operations. D = 1, then the string will be accessed from higher memory address to lower memory address, and if D = 0, it will do the reverse.
I	This is interrupt flag. If I = 1, then MPU will recognize the interrupts from peripherals. For I = 0, the interrupts will be ignored
T	This trap flag is used for on-chip debugging. When T = 1, it will work in a single step mode. After each instruction, one internal interrupt is generated. It helps to execute some program instruction by instruction.

- Assembly language consists of two types of statements viz.
- **Executable statements**- These are the statements to be executed by the processor. It consists of the entire instruction set of 8086.
- **Assembler directives**- These are the statements that direct the assembler to do something. As the name says, it directs the assembler to do a task.

8086 Instruction Set

- DATA TRANSFER INSTRUCTIONS-

1. MOV – MOV Destination, Source

MOV CX, 037AH

MOV BL, [437AH]

MOV AX, BX

MOV DL, [BX]

MOV DS, BX

MOV RESULT [BP], AX

MOV ES: RESULTS [BP], AX

8086 instructions do not permit memory to memory transfers or in general both the operands can not be in memory.

The immediate to register transfer is not permitted for segment registers.

Write a program to exchange two words stored in the memory.

```
data segment
    n1 dw 5555H
    n2 dw 6666H
data ends
code segment
    assume cs:code,ds:data
    mov ax,data
    mov ds,ax
    mov ax,n1
    mov bx,n2
    mov n1,bx
    mov n2,ax
    int 03
code ends
end
```

DATA TRANSFER INSTRUCTIONS(Contd)

- **XCHG – XCHG Destination, Source**
- **source and destination can not be memory location**

XCHG AX, DX

XCHG BL, BYTE PTR [SI]

XCHG AX,n

XCHG [SI+2],AX

- **LEA – LEA Register, Source**

LEA BX, n

LEA DI, [BX+SI]

LAHF: Load AH with lower byte of the flag.

Ah ← Lower byte of Flag register of the 8086

LAHF and SAHF

LAHF: Store AH to lower byte of the flag.

Lower byte of Flag register of the 8086 ← AH

DATA TRANSFER INSTRUCTIONS(Contd)

- **LDS – LDS Register, Memory address of the first word**

LDS BX, [4326]

LDS SI, n

BX ← [4326] and DS ← [4328]

- **LES – LES Register, Memory address of the first word**

LES BX, [789AH]

LES DI, [BX]

BX ← [789A] and ES ← [789C]

ARITHMETIC INSTRUCTIONS

- **ADD – ADD Destination, Source**
- **ADC – ADC Destination, Source**
- The source may be an immediate number, a register, or a memory location. The destination may be a register or a memory location. The source and the destination in an instruction cannot both be memory locations.
- Flags affected: AF, CF, OF, SF, ZF.

EX: ADD AL, 74H

ADC CL, BL

ADD DX, BX

ADD DX, [SI]

ADC AL, PRICES [BX]

ADD AL, PRICES [BX]

- **SUB – SUB Destination, Source**
- **SBB – SBB Destination, Source**
- Flags affected: AF, CF, OF, PF, SF, ZF.

Ex- SUB CX, BX

SBB CH, AL

SUB AX, 3427H

SBB BX, [3427H]

SUB PRICES [BX], 04H

SBB CX, TABLE [BX]

SBB TABLE [BX], CX

- **INC – INC Destination**

- AF, OF, PF, SF, and ZF are updated, but CF is not affected.
- Ex: INC BL
- INC CX
- INC BYTE PTR [BX]
- INC WORD PTR [BX]
- INC TEMP

DEC – DEC Destination

- This instruction subtracts 1 from the destination word or byte. The destination can be a register or a memory location. AF, OF, SF, PF, and ZF are updated, but CF is not affected.
 - DEC BYTE PTR [BX]
 - DEC WORD PTR [BP]
 - DEC COUNT

Sample program to add two numbers

data Segment

n1 db 25

n2 db 43

data ends

code segment

assume cs:code,ds:data

mov ax,data

mov ds,ax

mov al,n1

mov bl,n2

add ax,bx

int 03

code ends

end

CBW (CONVERT SIGNED BYTE TO SIGNED WORD)

This instruction copies the sign bit of the byte in AL to all the bits in AH. AH is then said to be the sign extension of AL. CBW does not affect any flag.

CWD (CONVERT SIGNED WORD TO SIGNED DOUBLE WORD)

This instruction copies the sign bit of a word in AX to all the bits of the DX register. In other words, it extends the sign of AX into all of DX. CWD affects no flags.

NEG destination

Destination can be register or memory location

Destination is replaced by 2's complement.

Neg AL

Neg word ptr [bx]

- **MUL – MUL Source**

$AX \leftarrow AL * \text{Source}$

$DX:AX \leftarrow AX * \text{Source}$

- Multiplies *unsigned* bytes. The source can be a register or a memory location.
- If the most significant byte of a 16-bit result or the most significant word of a 32-bit result is 0, CF and OF will both be 0's. Otherwise CF=OF=1. The conditional flag AF, PF, ZF and SF are undefined for this instruction.

Ex: MUL BH

MUL CX

MUL BYTE PTR [BX]

MUL FACTOR [BX]

MOV AX, MCAND_16

MOV CL, MPLIER_8

MOV CH, 00H

MUL CX

- **IMUL – IMUL Source**

multiplies a *signed* byte.

If the most significant byte of a 16-bit result or the most significant word of a 32-bit result is 0, CF and OF will both be 0's. Otherwise CF=OF=1. The conditional flag AF, PF,ZF and SF are undefined for this instruction.

Ex:IMUL BH

IMUL AX

DIV – DIV Source

Used to divide an *unsigned* word by a byte or to divide an *unsigned* double word (32 bits) by a word.

Ah(Rem):Al(Quo) $\leftarrow$ AX/Source

DX(Rem):AX(QUO) $\leftarrow$ (DX:AX)/Source

If an attempt is made to divide by 0 or if the quotient is too large to fit in the destination (greater than FFH / FFFFH), the 8086 will generate a type 0 interrupt. All flags are undefined after a DIV instruction.

Ex: DIV BL

DIV CX

IDIV – IDIV Source

- This instruction is used to divide a *signed* word by a *signed* byte, or to divide a *signed* double word by a *signed* word.

Ex: IDIV BL

IDIV BP

IDIV BYTE PTR [BX]

Logic (Bit Manipulation) Instructions

AND: Instruction

- AND Dest,Source
CF and OF are both 0 after AND. PF, SF, and ZF are updated by the AND instruction. AF is undefined. PF has meaning only for an 8-bit operand.
- MOV AL, 10101110b
AND AL, 11110110b
- AND CX, [SI]
- AND BH, CL
- AND BX, 00FFH

Logic (Bit Manipulation)

Instructions

OR Instruction

OR Dest,Source

CF and OF are both 0 after OR. PF, SF, and ZF are updated by the OR instruction. AF is undefined. PF has meaning only for an 8-bit operand.

OR BP, SI

OR SI, BP

OR BL, 80H

OR CX, TABLE [SI]

XOR – XOR Destination, Source

- XOR CL, BH
- XOR BP, DI
- XOR WORD PTR [BX], 00FFH
- CF and OF are both 0 after XOR. PF, SF, and ZF are updated.

NOT – NOT Destination

- NOT BX
- NOT BYTE PTR [BX]

CMP – CMP Destination, Source

- AF, OF, SF, ZF, PF, and CF are updated.
- CMP AL, 01H
- CMP BH, CL
- CMP CX, TEMP

TEST – TEST Destination, Source

- This instruction ANDs the byte / word in the specified source with the byte / word in the specified destination. Flags are updated, but neither operand is changed.
- TEST AL, BH
- TEST CX, 0001H
- TEST BP, [BX][DI]

INSTRUCTION FORMAT OF 8086

MOV instruction :Immediate to Register

1	0	1	1	W	REG	Data(W=0)	Data(W=1)
---	---	---	---	---	-----	-----------	-----------

Ex: MOV AX,1234h

This can be coded as

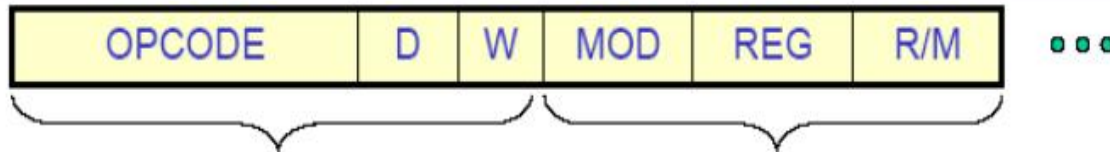
1 0 1 1 1 0 0 0 0 0 1 1 0 1 0 0 0 0 0 1 0 0 1 0
=> B8 34 12

MOV AL,12h

1 0 1 1 0 0 0 0 0 0 0 1 0 0 1 0=> B0 12

000	W=0	W=1
001	AL	AX
010	CL	CX
011	DL	DX
100	BH	SP
101	CH	BP
110	DH	SI
111	BH	DI

Converting Assembly Language Instructions to Machine Code



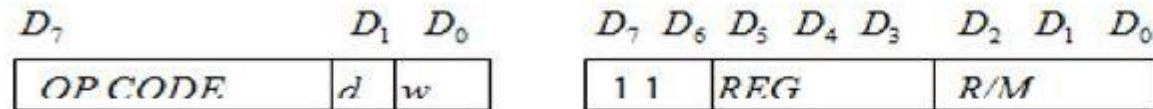
- An instruction can be coded with 1 to 6 bytes
- Byte 1 contains three kinds of information:
 - Opcode field (6 bits) specifies the operation such as add, subtract, or move
 - Register Direction Bit (D bit)
 - Tells the register operand in REG field in byte 2 is source or destination operand
 - 1: Data flow to the REG field from R/M
 - 0: Data flow from the REG field to the R/M
 - Data Size Bit (W bit)
 - Specifies whether the operation will be performed on 8-bit or 16-bit data
 - 0: 8 bits
 - 1: 16 bits

Byte 2 has three fields

- Mode field-2 bits
- Register field-3 bits
- Reg/Mem field-3 bits

MOD / R/M	Memory Mode (EA Calculation)			Register Mode	
	00	01	10	W=0	W=1
000	(BX)+(SI)	(BX)+(SI)+d8	(BX)+(SI)+d16	AL	AX
001	(BX) + (DI)	(BX)+(DI)+d8	(BX)+(DI)+d16	CL	CX
010	(BP)+(SI)	(BP)+(SI)+d8	(BP)+(SI)+d16	DL	DX
011	(BP)+(DI)	(BP)+(DI)+d8	(BP)+(DI)+d16	BL	BX
100	(SI)	(SI) + d8	(SI) + d16	AH	SP
101	(DI)	(DI) + d8	(DI) + d16	CH	BP
110	d16	(BP) + d8	(BP) + d16	DH	SI
111	(BX)	(BX) + d8	(BX) + d16	BH	DI

Register to Register : This format is 2 bytes long. The first byte of the code specifies the operation code and the width of the operand specifies by w bit. The second byte of the opcode shows the register operands and RIM field



The register represented by the REG field is one of the operands. The RIM field specifies another register or memory location, i.e., the other operand. The register specified by REG is a source operand if $D = 0$, else it is a destination operand.

For example:

MOV : data transfer operation from Register to Register.

Op-code is



1 0 0 0 1 0 0 0 1 1 0 0 0 0 0 1 88_H $C1_H$

$REG = 0 0 0$ indicates Register AL

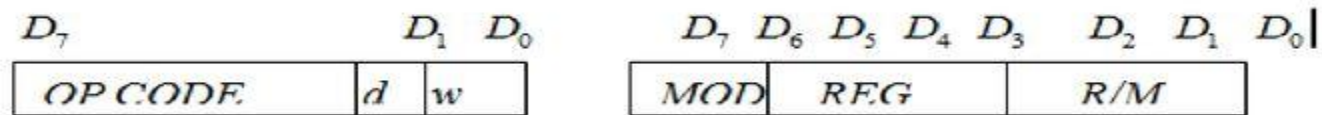
$REG = 0 0 1$ indicates Register CL

$w = 0$ indicates it is a byte operation (8 bit)

$d = 0$ indicates AL is a source register.

This instruction indicates MOV CL, AL , i.e. $CL \leftarrow AL$

Register to/from memory with no displacement : This format is also 2 bytes long and similar to the register to register format except for the *MOD* field.



The *MOD* field shows the *MOD* of addressing. In case of no displacement, $MOD = 00$

For example :

MOV : Data transfer Register/memory to/from register.



This format is similar to register to register transfer. The difference is in mood field.

For register to register, $mod = 11$

For register to/from memory with no displacement, $mod = 00$.

When $mod = 00$, the r/m fields indicates the address to memory location.

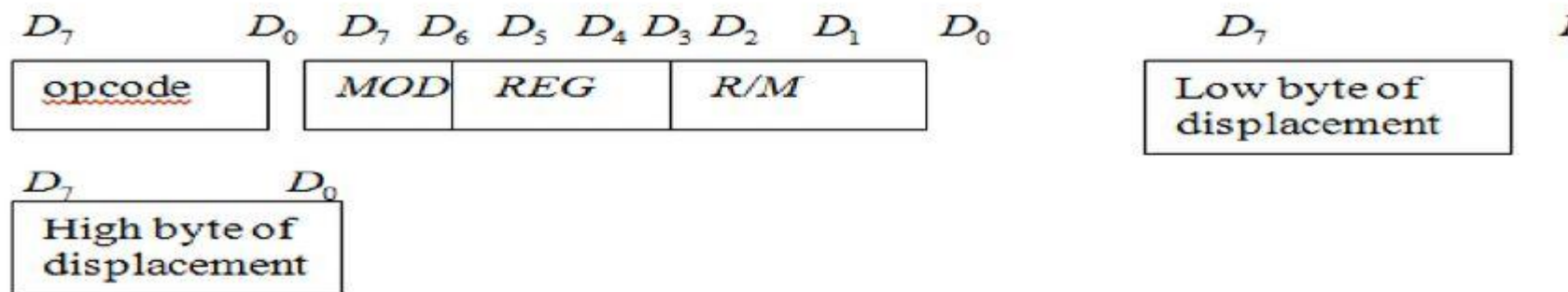
As for example $r/m = 111$ indicates (Bx)

The instruction

1 0 0 0 0 1 0 1 0 0 0 0 0 0 1 1 1 indicates the instruction *MOV AX, [BX]*

d) Register to/from Memory with Displacement :

This type of instruction format contains one or two additional bytes for displacement along with 2-byte the format of the register to/from memory without displacement.



$MOD = 0\ 1$ indicates displacement of 8 bytes (instruction is of size 3 bytes)

$MOD = 1\ 0$ indicates displacement of 16 bytes. (instruction is of size 4 bytes)

Already we have seen the other two options of MOD

$MOD = 1\ 1$ indicates register to register transfer

$MOD = 0\ 0$ indicates memory without displacement

In this case, R/M fields indicates a memory when MOD is not 1 1

$R/M = 1\ 1\ 1$ indicates (BX)

When $MOD = 0\ 1$, the offset address is $(BX) + D8$

When $MOD = 1\ 0$, the offset address is $(BX) + D16$

Segment Override Prefix

- The **Segment Override Prefix** says that if we want to use some other segment register than the default segment for a particular code, then it is possible.

MOV AX , [BX]

MOV AX, SS : [BX]

Format of Segment Override prefix byte is:

0 0 1 Seg.Reg 1 1 0

Seg.Reg	Code
---------	------

CS	01
----	----

DS	11
----	----

ES	00
----	----

SS	10
----	----

MOV AL,ES:[DI]

0 1 1 0 0 1 1 0 1 0 0 0 1 0 1 0 0 0 0 0 0 1 0 1 => 66 8A 05

Give the machine code for following instructions:

i. MOV SI,[4567h]

ii. MOV BX,ES:[1234h]

iii. MOV DI,6543h

- 8086 provides unconditional and conditional instructions
- The unconditional jump instruction takes the jump unconditionally
- Unconditional Jump
 - Intrasegment
 - Intersegment

- Jmp address
- Directly or indirectly: direct jump and indirect jump.
- In same segment :intrasegment jump➡ only IP will get changed
- Intra segment: Near jump(16 bit) and short jump(8 bit)
- In different segment: Intersegment jump (far jump)➡ IP and CS both get change.

0000000000000000110➔

- ____: 1231
- ____:IP: 1234
- ____:1:1235
- ____:2
- ____:3:1237
- Displacement by 8 bits
- -128 to 127
- Displacement by 16 bit
- -32768 to 32767
- Forward and backward jump
- Jmp next
- -----
- -----
- Next:_____
- -----
- -----
- Jmp next

Intrasegment Unconditional Jump

- Direct IP relative (Short or Near Jump)
- Indirect (Near Jump)

Direct:

$IP \leftarrow IP + 8\text{bit signed displacement (short jump)}$

$IP \leftarrow IP + 16\text{bit signed displacement (Near Jump)}$

Indirect:

$IP \leftarrow \text{Register (16 bit)}$

$IP \leftarrow \text{Memory word}$

- The Jump can be categorized as forward jump or backward jump.
Forward Jump: target instruction is after the jump instruction
Backward Jump: target instruction is before the jump instruction
target

•**Backward and Forward jumps**

Backward:

```
      L10:  
      .....  
      JMP   L10
```

Forward:

```
      JMP   L10  
      .....  
      L10:
```

Memory Content	Code
Cs:0000 B9 05 00	Mov CX, 05
Cs:0003 03 C3	Next:Add AX,BX
Cs: 0005 90	NOP
Cs:0006 90	NOP
Cs:0007 EB FA	JMP Next
Cs:0009	

- 0000 0000 0000 1001
- 1111 1111 1111 1010
- -----
- 0000 0000 0000 0011

- If displacement is 8 bit the range is -128 to 127 bytes
- -32768 to 32767 for 16 bit displacement
- Backward jump enables the assembler to decide whether the JMP instruction is coded 2 or 3 bytes as target instruction is known.
- Forward jump coded as 3 bytes because target instruction is unknown. If target instruction is encountered in 127 bytes then second reserved byte is converted to NOP.
- If you are sure that distance is in range of 8 bit then jump can be written with SHORT assembler directive

JMP SHORT next

- If the destination is in **the same code segment as the JMP instruction**, then only **the instruction pointer** will be changed to get the destination location. This is referred to as a *near jump*.
- If the destination for the jump instruction is in a segment with a name different from that of the segment containing the JMP instruction, then both the instruction pointer and the code segment register content will be changed to get the destination location. This referred to as a *far jump*.

Intrasegment Jump(IP relative)

- There are two types
 - **Direct:** The offset of the target instruction as displacement in the instruction

Opcode	Low displacement	High Displacement
--------	------------------	-------------------

IP← IP+ 8 bit/16 bit signed displacement

(If 8 bit-Short Jump and if 16 bit-Near Jump)

Ex: jmp label

- **Indirect:** The offset of the target instruction using 16 bit register or memory word

Opcode	MOD	100	R/M	Low byte offset	High byte offset
--------	-----	-----	-----	-----------------	------------------

- IP← Register(16 bit) / IP← Memory Word
- Ex: jmp **bx** / jmp word ptr **[BX]**

- If the target address displacement is 8 bit then the range of jump in both the directions are -- -128 to +127.
- If the target address displacement is 16 bit then the range of jump in both the directions are -32768 to +32767.
- The near jump covers the full segment size i.e 64KB

- In case of forward jump the distance of target is not known. Hence it causes problem to the assembler in coding the jmp as short or near(i.e to reserve 1 byte or 2 byte).
- If the forward jump is coded as short jump, then the additional instruction NOP is inserted to fill the gap.

For Ex: JMP SHORT next.

- Cs: 0000_____
- _____
- _____
- intrasegment:displacement of -6
- _____
- _____
- Cs:FFFF_____

Intersegment

Data segment:

addr DD 1234ffABh

Code segment:

Mov SI, offset addr

Jmp dword ptr [SI]

- 0400:4567 → AB

– 68 → FF

– 69 → 34

– 6A → 12

– SI=4567

FF	AB
----	----

Opcode_of_jump AB FF 34 12

CS: 1234 IP: FFAB

- Intersegment
- Opcode offset-low offset-high segment-low segment-high
- `Jmp dword ptr[si]` → jmp-opcode 00 00 ff ff
- SI(offset addr): 00
 - 00
 - Ff
 - FfFFFF:0000

Indirect intersegment jump

Opcode	MOD	101	R/M	Low byte offset	High byte offset
--------	-----	-----	-----	-----------------	------------------

Conditional Jump → all the conditional jumps are short jump → 8 bit displacement

- The following rules apply to the discussions presented in this section.
- The terms above and below are used when referring to the magnitude of unsigned numbers. For example, the number 00000111 (7) is above the number 00000010 (2), whereas the number 00000100 (4) is below the number 00001110 (14).
- The terms greater and less are used to refer to the relationship of two signed numbers. Greater means more positive. The number 00000111 (+7) is greater than the number 11111110 (-2), whereas the number 11111100 (-4) is less than the number 11110100 (-6).

The 8086 conditional jump instructions

- These are the instruction that will change the flow only when some conditions are met.

MNEMONIC	CONDITION TESTED	"JUMP IF ..."
JNBE	$(CF \text{ or } ZF) = 0$	above/not below nor equal
JNB	$CF = 0$	above or equal/not below
JNAE	$CF = 1$	below/not above nor equal
JBE	$(CF \text{ or } ZF) = 1$	below or equal/not above
JC	$CF = 1$	carry
JZ	$ZF = 1$	equal/zero
JNLE	$((SF \text{ xor } OF) \text{ or } ZF) = 0$	greater/not less nor equal
JGE	$(SF \text{ xor } OF) = 0$	greater or equal/not less
JL	$(SF \text{ xor } OF) = 1$	less/not greater nor equal
JLE	$((SF \text{ xor } OF) \text{ or } ZF) = 1$	less or equal/not greater
JNC	$CF = 0$	not carry
JNE	$ZF = 0$	not equal/not zero
JNO	$OF = 0$	not overflow
JNP	$PF = 0$	not parity/parity odd
JNS	$SF = 0$	not sign
JO	$OF = 1$	overflow
JPE	$PF = 1$	parity/parity equal
JS	$SF = 1$	sign

Above and below → unsigned numbers

- Ax=8
- Cx=5
- Cmp Ax, Cx

C=0 z=0

Z=1 → JE

Above is greater. JA /JNBE → (C OR Z)==0

Cf=0 → JAE/JNB

Below → JB/JNAE (cF=1)

Below and equal → (C OR Z)==1 JBE/JNA

Jump $\rightarrow$ greater/ less $\rightarrow$ signed numbers

- JG $((SF \text{ xor } OF) \text{ or } zf) == 0$
- JGE $(SF \text{ xor } OF) == 0$
- JL $(SF \text{ xor } OF) == 1$
- JLE $((SF \text{ xor } OF) \text{ or } zf) == 1$
- Cmp -5,4 $\rightarrow$ sub $\rightarrow$
- -5 $\rightarrow$ 1111 1011 (2's complement)
- 4 $\rightarrow$ 0000 0100
- cmp $\rightarrow$ (1111 1011)-(0000 0100) $\rightarrow$ (-5)+(-4)= 1111 0111 $\rightarrow$ sign flag 1 over flow 0
- S xor O will be 1
- It will jump on JL as expected.

JCXZ target_ins

- Mov cx,05

next: JCXZ over

...

...

Dec CX

JMP next

Over:

Write a program to find the factorial of a given 8 bit number.

- Note: the number is such that its factorial is less than or equal to FFFFh.
- Number is stored in data segment say n
- $n=5$.

data segment

n db 05

fact dw ?

data ends

Code-----

Mov ax,data

mov ds,ax

MOV AL, 01H

MOV CL, n

LOOP: MUL CL

DEC CL

JNZ LOOP

MOV res, AX

- code segment
- assume cs:code,ds:data
mov ax,data
- mov ds,ax
- mov al,01
- mov bl,n
- l1: mul bl
- dec bl
- jnz l1
- mov fact,ax
- int 03
- code ends

Find Fibonacci numbers first 10 numbers starting from 0 and 1

- Data----
- fib db 10 dup(?)
- n db 10
- -----
- Mov SI, offset fib
- mov al,00
- Mov byte ptr [si], al
- Inc si
- mov bl,01
- Mov byte ptr [si], bl
- Inc si
- mov cl,n
- Sub cl,02
- Next: add al,bl
- Mov dl, al
- Mov byte ptr [si],al
- Inc si
- Mov al,bl
- Mov bl, dl
- Dec cl
- Jnz next